1. a. What are the benefits of using Singleton pattern? Explain in brief. 4

b. Which design pattern provides us a way to reduce memory requirement? Explain how this memory optimization actually works? What other design pattern automatically comes with this design pattern implementation? 5

a)

Singleton

Pros

- You can be sure that a class has only a single instance.
- You gain a global access point to that instance.
- The singleton object is initialized only when it's requested for the first time.
- Controlled access to sole instance
- Permits a variable number of instances.

Cons

- Violates the Single Responsibility Principle. The pattern solves two problems at the time.
- The Singleton pattern can mask bad design, for instance, when the components of the program know too much about each other.
- The pattern requires special treatment in a multithreaded environment so that multiple threads won't create a singleton object several times.
- It may be difficult to unit test the client code of the Singleton because many test frameworks rely on inheritance when producing mock objects. Since the constructor of the singleton class is private and overriding static methods is impossible in most languages, you will need to think of a creative way to mock the singleton. Or just don't write the tests. Or don't use the Singleton pattern.

Singletons are Pathological Liars Drawbacks of the Singleton

2. a. What does encapsulation mean in OOP language? Why is encapsulation considered as a vital concept of OOP language? 4

b. "*Strategy changes the guts of an object whereas Decorator changes the skins of an object*" ---Explain this statement with appropriate reasoning. 5

a)

Encapsulation is one of the fundamental concepts in Object-Oriented Programming (OOP). It refers to the bundling of data (attributes) and methods (functions) that operate on the data into a single unit, typically a class. Encapsulation also involves restricting direct access to some of an object's components, which is often achieved using access modifiers like private, protected, and public.

Why Encapsulation is Vital:
Data Hiding: Encapsulation allows you to hide the internal state of an object and only expose a controlled interface. This prevents external code from directly modifying the object's data, which can lead to inconsistencies.
Modularity: By encapsulating related data and behavior, you create modular code that is easier to understand, maintain, and reuse.
Security: Encapsulation helps in protecting the integrity of the data by controlling how it is accessed and modified.
Flexibility and Maintenance: Encapsulated code is easier to maintain and modify. Changes to the internal implementation of a class do not affect the external code that uses the class, as long as the interface remains the same.

b)
The statement “Strategy changes the guts of an object whereas Decorator changes the skins of an object” highlights the difference between the Strategy and Decorator design patterns.

Strategy Pattern:
Purpose: The Strategy pattern is a behavioral design pattern that allows you to define a family of algorithms, encapsulate each one, and make them interchangeable. It lets the algorithm vary independently from clients that use it.
Guts of an Object: The Strategy pattern changes the **internal behavior or algorithm of an object**. For example, if you have a sorting algorithm, you can switch between different sorting strategies (e.g., QuickSort, MergeSort) without changing the context that uses the sorting algorithm.

Decorator Pattern:
Purpose: The Decorator pattern is a structural design pattern that allows behavior to be added to individual objects, either statically or dynamically, without affecting the behavior of other objects from the same class.
Skins of an Object: The Decorator pattern changes the **external behavior or appearance of an object by wrapping it with additional functionality.** For example, you can add new features to a text processing object (e.g., adding bold, italic formatting) without altering its core functionality.

Reasoning:

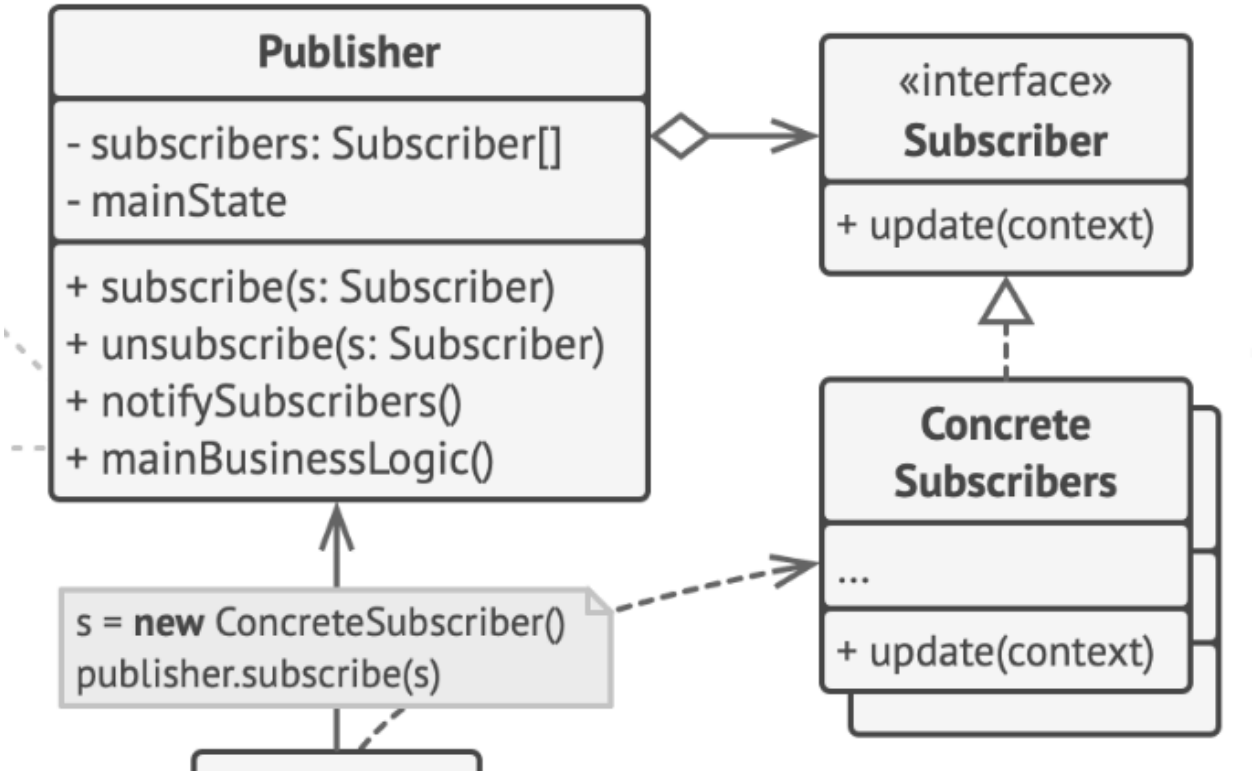
- Strategy focuses on changing the internal logic or algorithm of an object, hence "changing the guts."

- Decorator focuses on adding or modifying the external behavior or features of an object, hence "changing the skins."

In summary, encapsulation is crucial in OOP for data protection and modularity, while the Strategy and Decorator patterns serve different purposes in modifying object behavior, with Strategy altering internal logic and Decorator enhancing external features.

3. In an auction shop, an auctioneer accepts a bid from bidders. The auctioneer calls for a price for a product. When a bidder accepts a bid, he or she raises a numbered paddle which identifies the bidder. The bid price then changes and all the other bidders must be notified of the change. The auctioneer then broadcasts the new bid to the bidders. 12

Design the above mentioned system. Draw the class diagram and write the code to support your design.



Publisher -> Auctioneer
Subscriber -> Bidder

```
interface Bidder {
    void update(double bidPrice);
    void placeBid(Auctioneer auctioneer, double bid);
    String getName();
}
```

```
class Auctioneer {
```

```
    private List<Bidder> bidders = new ArrayList<>();
    private double bidPrice;

    public void registerBidder(Bidder bidder) {
        bidders.add(bidder);
    }

    public void removeBidder(Bidder bidder) {
        bidders.remove(bidder);
    }

    public void newBid(double bid, Bidder bidder) {
        System.out.println("Bidder " + bidder.getName() + " placed a new bid: $" + bid);
        bidPrice = bid;
        notifyBidders();
    }

    private void notifyBidders() {
        for (Bidder bidder : bidders) {
            bidder.update(bidPrice);
        }
    }
}


class ConcreteBidder implements Bidder {
    private String name;
    private int id;

    public ConcreteBidder(String name, int id) {
        this.name = name;
        this.id = id;
    }

    @Override
    public void update(double bidPrice) {
        System.out.println("Bidder " + name + " notified: New bid price is $" + bidPrice);
    }

    @Override
    public void placeBid(Auctioneer auctioneer, double bid) {
        auctioneer.newBid(bid, this);
    }
```

```
    @Override
    public String getName() {
        return name;
    }
}
```